第 1-1 讲 • 双击之后发生了什么

> **模块 1 • 程序到底是什么** ｜ 建议用时 **40 分钟**（阅读 25 + 动手 15）
> 前置要求：完成模块 0（会用终端、项目能跑起来）

本节目标

读完并做完这一讲，你能：

1. **说出"进程"是什么**，并能在任务管理器里指认出记账簿的进程。
2. 讲清"双击图标 → 程序运行"这条完整链路，包括**每一步是谁在干活**。
3. 理解**内存**是什么，以及为什么"关掉程序数据就没了"和"存到硬盘就还在"这两件事不一样。
4. 解释一个你自己遇到过、但可能没想明白的现象：**为什么点窗口的 X，记账簿没退出，而是躲到了右下角？**
5. 知道"程序卡死"和"程序崩溃"是两件不同的事，以及分别该怎么办。

一、核心概念：进程

1.1 现实比喻：一家餐厅的营业时间

把"程序"和"进程"分开想：

- **程序** = 一份菜谱。它躺在书架上，不会自己做菜。对应你硬盘上的 `记账簿.exe`。
- **进程** = 按照这份菜谱**真的开火营业**的那家店。有厨师在忙、有食材被消耗、有客人坐着。

关键区别：

	程序（菜谱）	进程（正在营业的店）
存在形式	硬盘上的一个文件	内存里正在运行的活物
关电脑后	还在	**消失**
能同时有几份	一份，可以复制多份	同一份菜谱可以同时开好几家店

这解释了一件事：为什么"双击打开一个软件"可以打开好几个？因为你把同一份菜谱，开了好几家店。每个店就是一个**进程**。

1.2 为什么需要"进程"这个概念

因为电脑要同时做很多事，得有人记账：**哪个任务用了多少内存、有没有在捣乱、要不要强行关掉。**

"进程"就是操作系统用来**管理"正在运行的任务"的单位**。任务管理器里那一行一行，每条就是一个进程。

1.3 内存是什么（这一步很关键）

硬盘是仓库：东西放进去，断电也在。但**取东西慢。** **内存**是操作台：取东西极快，但**一断电全没。**

所以程序运行时是这样的：

```
硬盘上的程序  →  被搬进内存  →  在内存里飞快地干活  →  需要保存时写回硬盘
（慢，但永久）                （快，但临时）              （慢，但永久）
```

这解释了两件事：

1. 为什么软件启动要等几秒？—— 它在把东西从硬盘搬进内存。
2. 为什么"没存盘就断电，工作就丢了"？—— 因为那部分成果还在内存里，没写回硬盘。

记账簿的设计正是围绕这一点：你点"保存"的那一刻，数据就被写进了硬盘上的数据库文件。所以哪怕电脑突然断电，你记的账也不会丢。（第 2-3 讲会讲这个机制。）

二、"双击之后"的完整链路

现在把概念串起来。你在桌面上双击「记账簿」图标，电脑内部发生的事：

步骤	谁在干活	发生了什么
1	**你**	双击图标（图标其实是一个"快捷方式"，指向真正的 exe）
2	**Windows 资源管理器**	收到双击，找到快捷方式指向的真实文件路径
3	**Windows 内核**	把它加载进内存，创建一个**新进程**
4	**记账簿的主进程**	启动，开始执行代码：读配置、开数据库、建窗口
5	**Chromium 引擎**	在主进程内部启动**渲染进程**，负责画界面（第 2-1 讲详讲）
6	**你**	看到窗口出现，可以开始记账

注意第 4、5 步：记账簿**不是一个进程，而是好几个。**这就是为什么任务管理器里你看到好几行「记账簿」——它们是一家人，各有分工。

2.1 你能验证这一点

打开记账簿，然后按 `Ctrl + Shift + Esc` 打开任务管理器，找「记账簿」。

你会看到不止一行。展开它们，会看到类似：

```
记账簿（主进程）
  └ 记账簿（渲染进程）
  └ 记账簿（GPU 进程）
  └ 记账簿（网络服务）
```

为什么这么多：这是 Electron 框架的设计——**让界面和"干实事的部分"分开住。**好处是界面崩了不至于连数据一起崩。这个设计是本项目最核心的架构决策，第 2-1 讲会专门讲透。

三、在项目里看真实文件

3.1 程序的"入口"

每个进程都有一个**起点**——代码从哪一行开始执行。记账簿主进程的起点是：

```
src\main\index.ts
```

打开它，你会看到类似这样的结构（简化）：

```
app.whenReady().then(() => {
  // 应用的"准备工作"都写在这里：
  // 注册协议、注册 IPC 通道、打开数据库、创建窗口……
})
```

`app.whenReady()` 的意思是"**等 Electron 准备好了，就执行下面的**"。这就是整个软件的"开机键"。

为什么入口这么重要：以后你要找某个功能在哪，最靠谱的方法就是**从入口顺着往下追**。（第 3-1 讲专门教这个"读代码心法"。）

3.2 快捷方式指向哪里

你桌面的「记账簿」图标是**快捷方式**（一个 `.lnk` 文件），不是程序本体。右键它 → 「属性」→ 看「目标」，你会看到它指向：

```
C:\Users\34123\AppData\Local\Programs\daily-ledger\记账簿.exe
```

这解释了一个常见困惑："我把桌面的图标删了，软件是不是没了？" —— 没有。删快捷方式只是删了一个路标，程序本体还在 `AppData` 里。真正的卸载要去「设置 → 应用」里卸载。

四、动手任务

步骤 1：看进程的真实样子

1. 打开记账簿（双击桌面图标）。
2. 按 `Ctrl + Shift + Esc` 打开任务管理器。
3. 在列表里找到「记账簿」，**点左边的小箭头展开**。
4. **数一数有几个进程**，把它们的类型记下来。

预期：至少 3–4 个。你会看到"主进程""渲染进程"这类字样（不同 Windows 版本显示略有差异）。

步骤 2：亲手验证"关掉进程会怎样"

这一步会用到你的记账簿，但不会损坏数据（数据在点保存时就写进硬盘了）。

1. 在任务管理器里，**只结束那个「渲染进程」**（选中它 → 右下角"结束任务"）。
2. **观察窗口**：窗口会变成空白/白屏，但进程列表里"主进程"还在。
3. 这就是"界面崩了，但程序没崩"——**因为它们是分开的进程。**
4. 把记账簿彻底关掉（任务管理器里结束主进程，或托盘图标 → 退出），重新打开，一切正常。

这一课的意义：你亲眼看到了"多进程架构"的实际效果。这是第 2-1 讲的理论基础。

步骤 3：搞清楚"X"到底干了什么

这是记账簿一个**用户最常困惑**的行为：

1. 打开记账簿。
2. 点窗口右上角的 **X**。
3. **看任务栏右下角**（Win11 可能收在 `^` 折叠区里）—— 找那个青绿色的账本小图标。
4. **左键单击**它 → 窗口回来了。

解释：点 X 不是"退出程序"，而是"把窗口藏进托盘"。**进程还活着**，只是没有窗口。

为什么这样设计：记账软件用户希望它一直待命（随手记一笔），而不是每次都重新启动。

想真正退出：右键托盘图标 → 「退出」。

步骤 4（思考题，不用写代码）

假设你在记账簿里记了一笔账，**没有点保存**，直接拔掉电源。

问题：这笔账会丢吗？

答案：会丢——但不是因为记账簿的问题，而是"没有提交的东西还在内存里"。等你做到第 2-3 讲，会亲眼看到"点保存"那一刻数据是怎么落到硬盘上的。

卡住了怎么办

现象	原因	怎么办
任务管理器里找不到"记账簿"	显示的是"应用"汇总视图	切到「详细信息」标签页，按名称排序
结束渲染进程后窗口全白，关不掉	界面进程没了，窗口是"尸体"	用任务管理器结束主进程，然后重新打开
托盘图标找不到	Win11 默认折叠	点任务栏的 `^`，把图标拖到常显区
点了 X 后任务栏图标也没了	正常现象，这是设计如此	去托盘里找

五、自测题

1. "程序"和"进程"有什么区别？关掉电脑后哪个会消失？

2. 内存和硬盘有什么不同？为什么软件启动要等几秒？

3. 为什么任务管理器里「记账簿」有好几行？

4. 你点了窗口的 X，记账簿真的退出了吗？怎么才能彻底退出？

5. 记账簿主进程的"入口文件"是哪个？为什么找到它很重要？

答案与解析

1. 程序是**硬盘上的静态文件**（菜谱），进程是**内存里正在运行的活物**（正在营业的店）。 → 关电脑后**进程消失，程序还在**。同一份程序可以同时开多个进程。

2. 硬盘**慢但永久**，内存**快但断电就没了**。程序启动时要把自己从硬盘搬进内存，这个搬运需要时间。 → 也解释了"没保存的改动会丢"——那部分还在内存里。

3. 因为记账簿用的是**多进程架构**（Electron）：主进程管数据与系统，渲染进程管界面，还有 GPU、网络等辅助进程。 → 核心好处：**界面出问题不会连带毁掉数据**。你在步骤 2 里亲眼验证了这一点。

4. 没有退出，只是把窗口藏进了系统托盘（右下角），进程仍在运行。 → 彻底退出：**右键托盘图标 → 退出**。这个设计是为了让记账软件随叫随到。

5. `src\main\index.ts`（主进程入口）。 → 它是整个软件的"开机键"，所有初始化动作都从那里开始。

读陌生代码时，从入口顺着往下追是最有效的方法。

六、术语表

术语	一句话解释
程序 / Program	硬盘上的可执行文件，静态的"菜谱"
进程 / Process	程序运行起来后在内存中的实例，"正在营业的店"
内存 / RAM	临时工作台：快，但断电全丢
硬盘 / Disk	永久仓库：慢，但断电还在
加载 / Load	把程序从硬盘搬进内存的过程
入口 / Entry	代码开始执行的那个文件（本项目是 `src/main/index.ts`）
快捷方式 / .lnk	指向真实程序的"路标"，删掉不影响程序本体
任务管理器	Windows 自带工具，查看和结束进程（`Ctrl+Shift+Esc`）
系统托盘	任务栏右下角那片小图标区域
多进程架构	把一个软件拆成多个进程协作，互不拖累
崩溃 / Crash	进程异常终止；**界面崩溃**和**主进程崩溃**后果不同
白屏	界面进程出问题的典型表现，窗口还在但画不出内容

七、本讲与项目的对应

- 你查看的文件：`src\main\index.ts`（主进程入口）
- 你做的操作：结束渲染进程、点 X 观察托盘行为 —— **不会损坏记账数据**

- 上一讲：0-2 装好工作台 ｜ 下一讲：**1-2 一切都是文件**（文件、路径、编码——为什么中文路径会出问题）

讲义版本：v1 ｜ 对应项目版本：0.1.0